

Actividad:

Herencia

Guevara Badillo Areli Alejandra
7CM1

Martes, 28 de abril, 2026

Código 1:

```
open class Libro(val titulo:String, val autor:String="desconocido", val ISBN:Long = 0)
class LibroFiccion(titulo:String, autor:String="desconocido"):Libro(titulo, autor)

open class Base(val x:Int, val tipo:Int, var mensaje:String=""){
    constructor(x:Int, mensaje:String=""):this(x,0,mensaje)
}
class Derivada(val enviar:Int, x:Int, tipo:Int, mensaje:String=""):Base(x, tipo, mensaje){
    constructor(enviar:Int, x:Int, msg:String=""):this(enviar, x, 0, msg)
}

fun main1(){
    println("=== PRUEBA PARTE 1 ===")

    // Probando Libro y LibroFiccion
    val libro1 = Libro("Kotlin for Dummies")
    val libro2 = LibroFiccion("1984", "George Orwell")
    println("Libro Base: Título: ${libro1.titulo}, Autor: ${libro1.autor}, ISBN: ${libro1.ISBN}")
    println("Libro Ficción: Título: ${libro2.titulo}, Autor: ${libro2.autor}, ISBN: ${libro2.ISBN}")

    println("-----")

    // Probando Base (usando constructor secundario que pone tipo en 0)
    val objBase = Base(10, "Mensaje Base")
    println("Base -> x: ${objBase.x}, tipo: ${objBase.tipo}, mensaje: ${objBase.mensaje}")

    // Probando Derivada (usando constructor secundario que pone tipo en 0)
    val objDerivada = Derivada(100, 5, "Mensaje Derivada")
    println("Derivada -> enviar: ${objDerivada.enviar}, x: ${objDerivada.x}, tipo: ${objDerivada.tipo}, mensaje: ${objDerivada.mensaje}")
    println()
}
```

Ejecución:

```
=== PRUEBA PARTE 1 ===  
Libro Base: Título: Kotlin for Dummies, Autor: desconocido, ISBN: 0  
Libro Ficción: Título: 1984, Autor: George Orwell, ISBN: 0  
-----  
Base -> x: 10, tipo: 0, mensaje: Mensaje Base  
Derivada -> enviar: 100, x: 5, tipo: 0, mensaje: Mensaje Derivada
```

Código 2:

```
open class base(val mensaje:String, val correo:String){  
    init {  
        println("Init en class Base")  
    }  
    constructor(correo:String):this("Sin msg", correo){  
        println("ConstructorSecundario class Base")  
    }  
}  
class derivada(correo:String):base(correo){  
    init{  
        println("Init class derivada")  
    }  
    constructor():this("micorreo@gmail.com"){  
        println("Constructor secundario class derivada")  
    }  
}  
  
fun main2(){  
    println("=== PRUEBA PARTE 2 ===")  
  
    println("-> Creando instancia de derivada usando constructor secundario (sin  
parámetros):")  
    // Esto disparará una cadena de llamadas a constructores e inits hacia arriba  
    val d1 = derivada()  
    println("Valores finales d1 -> Correo: ${d1.correo}, Mensaje: ${d1.mensaje}")  
  
    println("-----")  
  
    println("-> Creando instancia de derivada usando constructor primario:")  
    val d2 = derivada("usuario@empresa.com")  
    println("Valores finales d2 -> Correo: ${d2.correo}, Mensaje: ${d2.mensaje}")  
    println()  
}
```

Ejecución:

```
=== PRUEBA PARTE 2 ===
-> Creando instancia de derivada usando constructor secundario (sin parámetros):
Init en class Base
ConstructorSecundario class Base
Init class derivada
Constructor secundario class derivada
Valores finales d1 -> Correo: micorreo@gmail.com, Mensaje: Sin msg
-----
-> Creando instancia de derivada usando constructor primario:
Init en class Base
ConstructorSecundario class Base
Init class derivada
Valores finales d2 -> Correo: usuario@empresa.com, Mensaje: Sin msg
```

Código 3:

```
open class Figura{
    open val vertices:Int=0
    open fun Dibuja(){
        println("Dibujar figura")
    }
    fun relleno(){
        println("Rellena figura")
    }
}
class Circulo():Figura(){
    override fun Dibuja(){
        println("Dibuja circulo")
    }
}

fun main3(){
    println("=== PRUEBA PARTE 3 ===")

    val figuraNormal = Figura()
    print("Llamando a Figura normal: ")
    figuraNormal.Dibuja()

    val miCirculo = Circulo()
    print("Llamando a Circulo: ")
    miCirculo.Dibuja() // Usa el método sobrescrito

    print("Llamando a método heredado desde Circulo: ")
    miCirculo.relleno() // Usa el método de la clase padre
    println()
}
```

Ejecución:

```
=== PRUEBA PARTE 3 ===  
Llamando a Figura normal: Dibujar figura  
Llamando a Circulo: Dibuja circulo  
Llamando a método heredado desde Circulo: Rellena figura
```

Código 4:

```
open class Rectangulo():Figura(){  
    override val vertices = 4  
    open fun dibuja(){ // Nota: Este es un método nuevo, diferente a Dibuja() por la minúscula  
        println("Dibuja Rectangulo")  
    }  
    val colorBorde:String  
    get()="negro"  
}  
class RectanguloRelleno:Rectangulo(){  
    override fun dibuja(){  
        super.dibuja()  
        println("Dibuja rectangulo relleno")  
    }  
    val relleno:String  
    get() = super.colorBorde  
}  
  
fun main4(){  
    println("=== PRUEBA PARTE 4 ===")  
  
    val rect = Rectangulo()  
    println("Rectángulo vértices: ${rect.vertices}")  
    print("Método dibuja() (minúscula) de Rectángulo: ")  
    rect.dibuja()  
  
    println("-----")  
  
    val rectRelleno = RectanguloRelleno()  
    println("Llamando a dibuja() en RectanguloRelleno (llama a super y luego imprime su propio texto):")  
    rectRelleno.dibuja()  
    println("Color del borde del padre: ${rectRelleno.colorBorde}")  
    println("Propiedad 'relleno' obtenida del padre: ${rectRelleno.relleno}")  
    println()  
}
```

Ejecución:

```
=== PRUEBA PARTE 4 ===
Rectángulo vórtices: 4
Método dibuja() (minúscula) de Rectángulo: Dibuja Rectángulo
-----
Llamando a dibuja() en RectánguloRelleno (llama a super y luego imprime su propio texto):
Dibuja Rectángulo
Dibuja rectángulo relleno
Color del borde del padre: negro
Propiedad 'relleno' obtenida del padre: negro
```